

DAY 2 — COMPLETE EMBEDDED LINUX SYSTEM BUILD (OS CREATION)

MAIN GOAL

Build a complete ARM Embedded Linux firmware system containing:

Component	Purpose
Linux Kernel	Core operating system
BusyBox	Lightweight Linux utilities
RootFS	Embedded Linux filesystem
Toolchain	ARM cross compiler
Device Tree	Hardware description
Boot Image	QEMU bootable system

Final Result:

Boot complete ARM Embedded Linux
inside QEMU
WITHOUT real hardware

COMPLETE END-TO-END FLOW

Ubuntu x86 Machine



Buildroot Build System



Downloads Toolchain





Builds Linux Kernel



Builds BusyBox



Creates Root Filesystem



Generates Boot Images



Launches ARM Linux in QEMU

STEP 1 — INSTALL DEPENDENCIES

`sudo apt update`

PURPOSE

Updates Ubuntu package database.

WHY THIS IS NEEDED

Ubuntu needs latest package metadata before installing tools.

INTERNAL FLOW

Ubuntu package manager



Connects to package repositories



Downloads latest package indexes

INSTALL REQUIRED PACKAGES

```
sudo apt install build-essential git cpio unzip rsync bc bison flex libssl-dev make  
qemu-system-arm
```

MAIN PURPOSE

Installs tools required to build Embedded Linux firmware.

PURPOSE OF EACH PACKAGE

Package	Purpose	Why Needed
<code>build-essential</code>	GCC compiler + build tools	Compile Linux packages
<code>git</code>	Source control tool	Clone Buildroot source
<code>cpio</code>	Archive utility	Build root filesystem
<code>unzip</code>	ZIP extraction tool	Extract downloaded packages
<code>rsync</code>	File synchronization	Copy filesystem efficiently
<code>bc</code>	Calculator utility	Linux kernel build dependency
<code>bison</code>	Parser generator	Required for kernel compilation
<code>flex</code>	Lexical parser	Used in build systems
<code>libssl-dev</code>	SSL development library	Secure package downloads
<code>make</code>	Build automation tool	Executes build system
<code>qemu-system-arm</code>	ARM hardware emulator	Runs ARM Linux system

WHAT THIS STEP ACHIEVES

Your Ubuntu becomes:

Embedded Linux Build Machine

Capable of:

- Building Linux kernel
 - Building BusyBox
 - Creating root filesystem
 - Running ARM virtual hardware
-

VERIFY QEMU INSTALLATION

`qemu-system-arm --version`

PURPOSE

Checks ARM system emulator installation.

THINKING FLOW

Question

Can Ubuntu emulate complete ARM hardware?

Answer

QEMU installed successfully

OUTPUT

Displays QEMU version.

STEP 2 — CREATE WORKSPACE

```
mkdir -p ~/ilo-training
```

PURPOSE

Creates training/project directory.

WHY THIS IS NEEDED

Buildroot generates:

- Kernel files
- Toolchains
- RootFS
- Output images
- Temporary build files

So dedicated workspace is required.

MOVE INTO WORKSPACE

```
cd ~/ilo-training
```

PURPOSE

Switch terminal into workspace directory.

STEP 3 — CLONE BUILDROOT

```
git clone https://github.com/buildroot/buildroot.git
```

THIS IS THE MOST IMPORTANT CONCEPT

WHAT IS BUILDROOT?

Buildroot is:

Embedded Linux OS Generator

It automatically builds:

Component	Buildroot Creates
Linux Kernel	ARM kernel
BusyBox	Linux commands
RootFS	Embedded filesystem

Toolchain

ARM compiler

Boot images

QEMU bootable image

WHY BUILDROOT IS IMPORTANT

Without Buildroot:

You must manually:

- Build toolchain
- Build kernel
- Build BusyBox
- Create rootfs
- Create init scripts
- Configure boot arguments
- Debug kernel panics

Very difficult for beginners.

WHAT HAPPENS INTERNALLY

Git client



Connects GitHub



Downloads Buildroot source



Creates buildroot directory

PURPOSE OF EACH PART

Part

Meaning



`git clone` Download Git repository

`https://...` Buildroot source URL

`buildroot.g
it` Embedded Linux build framework

OUTPUT

`buildroot/`

STEP 4 — ENTER BUILDROOT

`cd buildroot`

PURPOSE

Moves terminal into Buildroot source tree.

WHY THIS IS IMPORTANT

Buildroot build system exists here.

Important directories include:

Directory	Purpose
<code>configs/</code>	Board configurations
<code>package/</code>	Package definitions

`system/` Root filesystem
 templates

`output/` Generated images

`toolchain/` Compiler build logic

STEP 5 — LOAD ARM QEMU CONFIGURATION

```
make qemu_arm_versatile_defconfig
```

THIS IS ONE OF THE MOST IMPORTANT COMMANDS

MAIN PURPOSE

Loads predefined ARM QEMU board configuration.

THINKING FLOW

Question

How does Buildroot know:

- which CPU?
- which board?

- which kernel?
- which rootfs?

Answer

Defconfig

WHAT THIS COMMAND DOES

Automatically configures:

Component	Configuration
ARM board	VersatilePB
Linux kernel	ARM kernel
BusyBox	Embedded Linux commands
RootFS	Embedded filesystem
QEMU	ARM virtual machine

INTERNAL FLOW

Buildroot config system



Loads qemu_arm_versatile_defconfig



Creates .config



System ready for ARM build

WHAT IS GENERATED?

.config

This file contains:

- Kernel settings
 - Filesystem settings
 - Toolchain settings
 - QEMU settings
-

VERY IMPORTANT UNDERSTANDING

This single command avoids manual configuration of:

- Kernel
 - RootFS
 - BusyBox
 - Bootloader
 - Device tree
 - QEMU arguments
-

STEP 6 — BUILD COMPLETE EMBEDDED LINUX SYSTEM

make -j\$(nproc)

THIS IS THE FULL OS BUILD STEP

MAIN PURPOSE

Builds complete ARM Embedded Linux firmware.

WHAT DOES \$(nproc) MEAN?

\$(nproc)

Automatically detects CPU cores.

Example:

8-core machine



make -j8

WHY THIS IS USED

Enables parallel compilation for faster builds.

WHAT HAPPENS INTERNALLY

Buildroot



Downloads source packages



Builds ARM toolchain



Builds Linux kernel



Builds BusyBox





Creates root filesystem



Creates device tree



Generates bootable images

COMPLETE INTERNAL BUILD FLOW

Buildroot Source



ARM Toolchain



Linux Kernel Build



BusyBox Build



RootFS Creation



Device Tree Build



QEMU Boot Image

IMPORTANT NOTE

Build may take:

15–40 minutes

depending on:

- CPU speed
 - Internet speed
 - RAM
 - SSD/HDD
-

SUCCESSFUL BUILD OUTPUT

Executing post-image script

Means:

Embedded Linux image generation SUCCESS

STEP 7 — VERIFY GENERATED IMAGES

ls output/images

PURPOSE

Displays generated bootable firmware files.

EXPECTED OUTPUT

rootfs.ext2
start-qemu.sh
versatile-pb.dtb
zImage

PURPOSE OF EACH FILE

File	Purpose
<code>rootfs.ext2</code>	Embedded Linux filesystem

`start-qemu.sh` Auto-generated QEMU launch script

`versatile-pb.dtb` ARM hardware description

`zImage` ARM Linux kernel image

VERY IMPORTANT UNDERSTANDING

At this stage you successfully built:

Complete ARM Linux Operating System

STEP 8 — BOOT EMBEDDED LINUX

`./output/images/start-qemu.sh`

THIS IS THE MOST IMPORTANT EXECUTION STEP

PURPOSE

Launches ARM Embedded Linux inside QEMU.

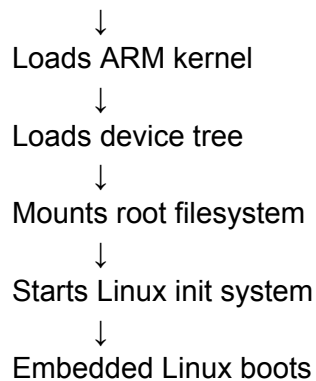
WHY THIS METHOD IS BEST

Buildroot already generated:

Item	Automatically Configured
Kernel args	✓
RootFS mount	✓
Device tree	✓
QEMU machine type	✓
Console settings	✓

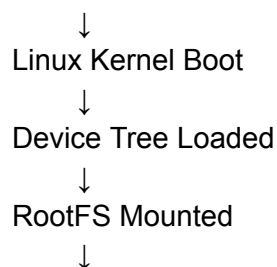
WHAT HAPPENS INTERNALLY

QEMU starts virtual ARM hardware



COMPLETE BOOT FLOW

QEMU ARM Hardware





BusyBox Init Starts



Linux Login Prompt

STEP 9 — LOGIN

When you see:

buildroot login:

Type:

root

If password appears:

Press ENTER

PURPOSE

Logs into Embedded Linux system.

EXPECTED OUTPUT

#

This means:

Linux shell access SUCCESS

STEP 10 — VERIFICATION COMMANDS

CHECK KERNEL

`uname -a`

PURPOSE

Displays kernel information.

CHECK CPU

`cat /proc/cpuinfo`

PURPOSE

Displays ARM CPU details.

CHECK RUNNING PROCESSES

`ps`

PURPOSE

Shows running Linux processes.

CHECK FILESYSTEMS

mount

PURPOSE

Shows mounted filesystems.

CHECK MEMORY

free -h

PURPOSE

Displays RAM usage.

CHECK NETWORK

ifconfig

PURPOSE

Displays network interfaces.

STEP 11 — SHUTDOWN SYSTEM

poweroff

PURPOSE

Gracefully shuts down Embedded Linux system.

WHAT HAPPENS INTERNALLY

Stops processes



Unmounts filesystems



Flushes buffers



Kernel shutdown



QEMU exits

COMPLETE SYSTEM ARCHITECTURE

Ubuntu x86 Host



Buildroot



ARM Toolchain



Linux Kernel



BusyBox



RootFS



↓
QEMU ARM Emulator
↓
Embedded Linux Boot

WHY BUILDROOT IS BEST FOR BEGINNERS

Buildroot avoids manual debugging of:

Manual Problem	Buildroot Fixes
Missing /init	✓
Broken rootfs	✓
Kernel panic	✓
Missing device nodes	✓
BusyBox issues	✓
Console binding issues	✓
PID1 crashes	✓

FINAL LEARNING OUTCOME

After this exercise, you understand:

Concept	Meaning
Embedded Linux	Complete ARM operating system

Buildroot	Embedded OS generator
RootFS	Linux filesystem
BusyBox	Embedded Linux utilities
QEMU	ARM hardware emulation
Device Tree	Hardware configuration
Linux Kernel	Core operating system
Toolchain	ARM compiler ecosystem

FINAL INDUSTRY UNDERSTANDING

This workflow is the foundation of:

- Embedded Linux engineering
- iLO/BMC firmware
- Automotive Linux systems
- IoT firmware
- Networking appliances
- ARM platform engineering
- Linux BSP development
- Firmware product engineering